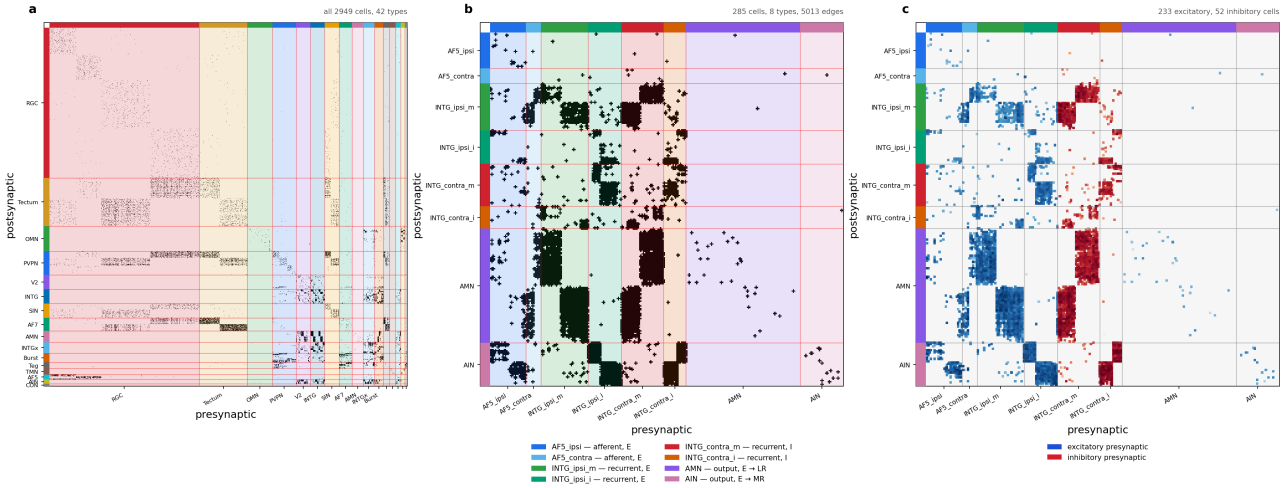
Research notebook

The larval zebrafish oculomotor integrator: circuit, task, and training data

Working document for the `feat/oculomotor` branch. It records what the circuit is taken to be, what the network will be asked to compute, and how the training data is generated. Everything here that is a *claim about biology* rather than a fact read off the reconstruction is marked as such — those are the parts that need the circuit owner's signature before any result means anything.

Source reconstruction: `Oculomotor_sortedData_081126.pkl`, 2949 cells, 42 cell types, 44,584 edges (density 0.51 %). Selected sub-circuit: `config/zebrafish/zebrafish_om_intg_285_v1.yaml`, 285 cells, 8 types.



The oculomotor reconstruction and the sub-circuit modelled here. (a) All 2949 cells, ordered by the 16 coarse cell-type families, support mask of the synapse-area matrix. (b) The 285-cell sub-circuit, ordered afferent then recurrent then output, and within a type left hemisphere before right. Colour carries the biology: blue = afferent, green = excitatory recurrent, red/orange = inhibitory recurrent, purple/pink = motor output. (c) The same sub-circuit signed by the Dale assignment of section 1.4 — each synapse coloured by the sign of its *presynaptic* cell, blue excitatory, red inhibitory, shade log-scaled in synaptic area. This is what the model multiplies, $\hat{W}_{ij} = |\hat{S}_{ij}|\,\mathrm{sign}(W^{\mathrm{con}}_{ij})$, and the support mask of (b) cannot show it. Because the sign belongs to the column, the panel reads as vertical bands: the two `INTG_contra` types are a solid red block, and it is visibly the only inhibition the 285 cells contain. Rows are postsynaptic, columns presynaptic in all three. Regenerate with `python scripts/plot_oculomotor_connectome.py -config config/zebrafish/zebrafish_om_intg_285_v1.yaml -out figures/zebrafish/fig_oculomotor_circuit.png`.

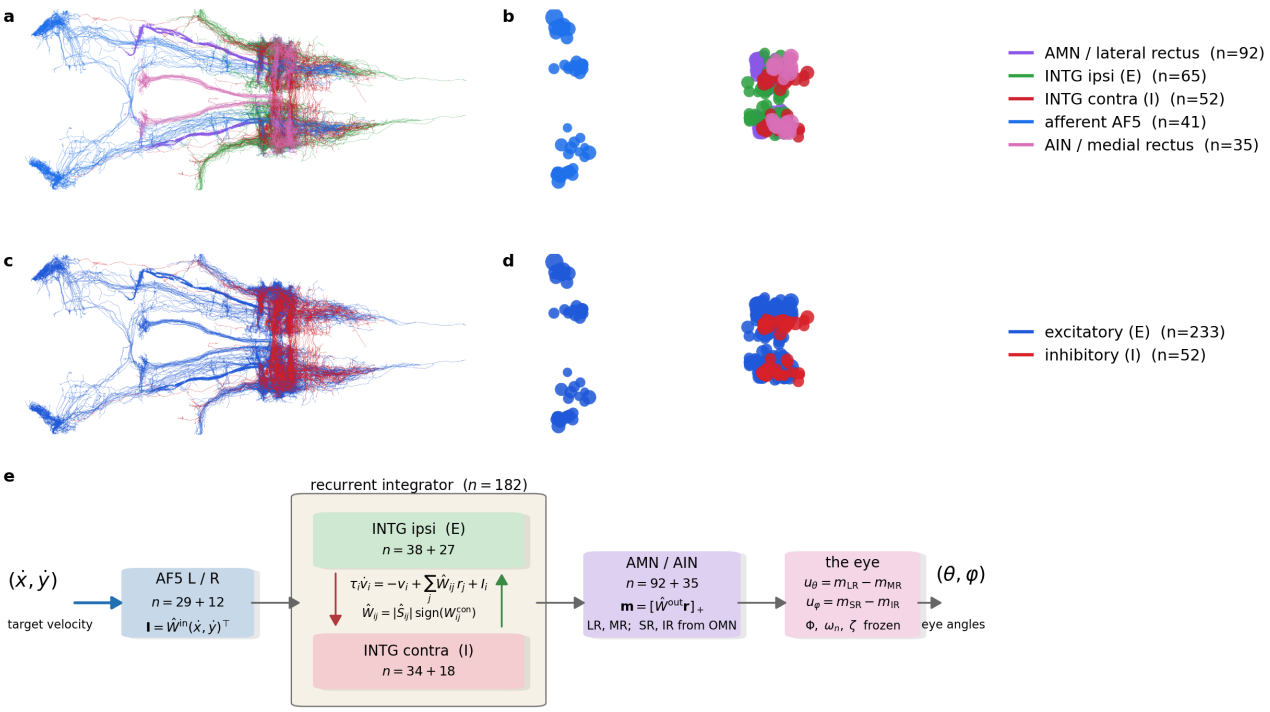


Figure 1 — the oculomotor circuit. Twin of Figure 1 of the zebrafish heading-direction paper, rendered through the same code with the content swapped. **(a)** All 285 skeletons from the neuprint-fish2 reconstruction, dorsal view, coloured by role: AF5 afferents blue, ipsilateral (excitatory) INTG green, contralateral (inhibitory) INTG red, AMN purple, AIN pink. **(b)** The cell bodies alone, same view — the AF5 somas sit well anterior of the integrator/motor cluster (radii scaled x0.3 for legibility, not a measurement). **(c, d)** The same skeletons and somas recoloured by Dale sign: blue excitatory (233 cells), red inhibitory (52 cells). The inhibitory population is exactly the contralateral integrator of section 1.4, and the two rows together are the claim being made — that the E/I split is by projection laterality, not by anatomical position. **(e)** The computation of section 5 end to end: the target velocity $(\dot{x}, \dot{y})$ enters through $\hat{W}^{\text{in}}$ on the AF5 afferents, the INTG pool integrates under sign-locked continuous-time rate dynamics with mutual inhibition across the midline, $\hat{W}^{\text{out}}$ reads non-negative motor drives, and the push-pull differences u_θ, u_φ drive the eye to the gaze angles (θ, φ). LR and MR have a pool in these 285 cells; SR and IR would come from OMN, which section 1.5 leaves out. Regenerate with `python figures/zebrafish/fig_1_oculomotor_overview.py`.

1. The circuit

1.1 Three pools

Anatomy for all 285 cells was fetched from neuprint-fish2 and is shown in Figure 1. It had to be keyed on bodyId rather than cell type: the server does not carry the reconstruction's type names — `INTG_ipsi_m` is `INTGip1` there, `INTG_contra_m` is `INTGco1` — and the two AF5 populations carry no type at all, so a type query returns 142 of 285 cells and drops the whole afferent stage. All 285 bodyIds resolve.

The 285 cells divide into an input stage, a recurrent integrator and a motor output stage.

pool	types	cells	Dale sign
afferent	`AF5_ipsi` (29), `AF5_contra` (12)	41	excitatory
recurrent	`INTG_ipsi_m` (38), `INTG_ipsi_i` (27), `INTG_contra_m` (34), `INTG_contra_i` (18)	117	ipsi excitatory, contra inhibitory
output	`AMN` (92), `AIN` (35)	127	excitatory

Within the sub-circuit there are 5013 edges, a density of 6.2 % — an order of magnitude above the 0.51 % of the full reconstruction. That concentration is what one expects of a recurrent integrator core plus its dedicated input and output stages, and it is the first (weak) evidence that the selection is a circuit rather than an arbitrary subset.

1.2 AF5 is an afferent, and the wiring says so

The two AF5 populations are the pretectal arborization field that carries optokinetic drive. Their afferent status is not assumed. In the measured matrix, AF5 sends 88.2 % of its outgoing synaptic area into the INTG/AMN/AIN core and receives 1/178th as much back (6.89e5 out against 3.87e3 in). This is the same asymmetry test that confirmed the matrix orientation using the retinal ganglion cells, which are 15.4x more outgoing than incoming.

Both AF5 types are left/right balanced — `AF5_ipsi` 15 L / 14 R, `AF5_contra` 6 L / 6 R — so a bilateral input gate is expressible. This is worth stating because it is not generally true in this dataset: `RGC_AF7` is reconstructed in the left hemisphere only (804 L / 0 R) and could not support a symmetric gate.

1.3 What drives the afferents — CLAIM, and an open one

The direction selectivity below is the circuit owner's description, not a measurement in this file. The combination rule is explicitly unresolved.

- `AF5_ipsi` — the left pool is driven by rightward-eye motion that is leftward and/or forward; the right pool by leftward-eye motion that is rightward and/or forward.
- `AF5_contra` — the left pool is driven by rightward-eye motion that is rightward and/or backward; the right pool by leftward-eye motion that is leftward and/or backward.

Whether the two conditions combine as AND, OR or XOR is **not known**. This is not a detail to be settled by a default: it decides whether a single afferent unit reports a conjunction (a narrow direction tuning) or a disjunction (a broad one), and therefore how much of the direction computation the recurrent circuit has to do itself. Until it is resolved, the input model should carry it as an explicit switch and the three variants should be trained and compared.

1.4 The integrator, and why the E/I split is by laterality

The four INTG types are the velocity-to-position integrator. The sign assignment follows their projection laterality: **ipsilateral projections excitatory, contralateral projections inhibitory**. That is the classical integrator motif — two half-integrators, one per hemisphere, each self-exciting locally and inhibiting its opposite number across the midline. Mutual inhibition is what lets the pair hold a *signed* position variable on a single positive-rate substrate. Panels (c) and (d) of Figure 1 plot the assignment on the anatomy — blue excitatory, red inhibitory — and what they show is that it is not readable from position: the 52 inhibitory cells are interdigitated with the 233 excitatory ones in the same hindbrain column, which is why the split has to be asserted per cell type rather than inferred from where a soma sits.

This is a Dale assignment by cell type, exactly as in the heading-direction circuit, where `_ZHD_INH_PREFIXES` in `connectome_loaders.py` encodes the claim that the r1pi/dIPN cells are GABAergic. Neither is a measurement. The difference is that here the claim is written in the config, per type, where it can be reviewed and flipped, rather than in a tuple of string prefixes inside a loader.

1.5 The output stage, and one simplification worth flagging

Only the **horizontal** pair of extraocular muscles is modelled:

Throughout, **the eye** means everything downstream of the last neuron — the globe, its six muscles, the orbital tissue and their mechanics. What matters is the boundary rather than the word: the circuit's output is a muscle command, the eye turns that command into an angle, and the eye is measured and frozen while the circuit is learned. The engineering literature, and several of the sources cited later, call this the *plant*; it is the same thing. Section 5.1 identifies it.

- `AMN` drives the **lateral rectus** (`LR` in the eye model), which abducts the eye;
- `AIN` drives the **medial rectus** (`MR`), which adducts it.

Muscle keys refer to the six-muscle soft-body eye in `Plexus/prototype/eye`, where each muscle carries an innervation state `muscle.act` and the globe's pose is recovered by a Kabsch fit (`eye_pose` -> horizontal, vertical, torsion in degrees). Coupling the readout to that eye, rather than to an abstract scalar, is what would make "eye position" a mechanical quantity instead of a fitted one.

The simplification. `AIN` are abducens *internuclear* neurons. In vivo they are not motor neurons: they project across the midline to the oculomotor nucleus (`OMN`), whose motor neurons innervate the medial rectus. `OMN` is 207 cells in this reconstruction and is **not** in the selected pool. Treating `AIN` as driving the medial rectus therefore collapses a two-synapse pathway into one, and drops the sign inversion and delay that the missing synapse would contribute. This is a legitimate modelling choice, but it should be stated in any write-up, and the obvious control is to add `OMN` to `circuit.cell_types` and check whether the fitted dynamics change.

1.6 What is not in the pool

Two omissions are deliberate and one of them constrains the task:

- `Burst_T1A/T1B/T8/T9/T10` (74 cells) are the saccadic burst generators — the source of the brief velocity command that a saccadic integrator integrates. They are excluded, so in the current pool the only anatomically defined input is the optokinetic AF5 drive. A saccade-driven task would have to inject velocity directly into INTG, which is a modelling choice, not a connectome-derived pathway.
- `OMN` (207 cells), as described in §1.5.

2. The task

2.1 What the network computes

The oculomotor integrator converts an eye-**velocity** command into an eye- **position** command: the motor neurons must hold a rate proportional to position long after the velocity signal has gone. Written as the leaky integral of the drive, per axis,

$$\frac{d\theta}{dt} = -\frac{\theta}{\tau} + s_\theta\,\dot{x}(t), \qquad \frac{d\varphi}{dt} = -\frac{\varphi}{\tau} + s_\varphi\,\dot{y}(t)$$

where (θ, φ) are the horizontal and vertical eye angles and $(\dot{x}, \dot{y})$ the target velocity — the notation of section 5, in which v is reserved for membrane voltage. The quantity of interest is τ, the time constant over which eye position leaks back to centre. A perfect integrator is $\tau = \infty$; the biological integrator is finite, and measuring what τ the connectome-constrained circuit can support is the point of the exercise.

2.2 Saccades versus optokinetic drive

The two natural stimulus regimes differ, and the choice interacts with §1.6:

- **Optokinetic**: sustained whole-field motion, the drive AF5 actually carries. Slow, continuous velocity — the regime the selected pool supports anatomically.
- **Saccadic**: brief high-velocity pulses separated by fixations, the classical integrator probe. This is the regime the burst generators serve, and they are not in the pool.

The swim generator's stimulus is a Poisson train of boxcar impulses (`swim_rate_hz`, `swim_duration_s`) — structurally a saccade train already. So the saccadic regime is reachable by reparameterising the existing generator rather than writing a new one; what is missing is anatomy for the input, not code.

2.3 The open-loop problem

Coarsely: the circuit is asked to keep a moving target centred on the fovea while being told only how fast the target is moving, never where it is. That is the situation the anatomy puts it in. AF5 is a motion-sensitive arborization field — it reports optokinetic *velocity* — and nothing in the selected 285-cell pool reports absolute target position. A controller with position feedback can always correct, so its errors do not accumulate; a controller given velocity alone must reconstruct position by integration and has nothing to check the result against. Drift is therefore not a failure of the circuit but a property of the problem, and the meaningful question is not whether the target is held but for how long.

More specifically, take the horizontal axis and write the target eccentricity $\theta^\star(t)$, the gaze $\theta(t)$, and the retinal error $\varepsilon = \theta^\star - \theta$; the vertical axis is the same with φ. Every trial starts foveated at the centre, $\theta(0) = \theta^\star(0) = 0$. The eye here is idealised as rate control: the motor command sets gaze *velocity*, so gaze angle is its integral. The ideal controller is then

$$\frac{d\theta}{dt} = \dot\theta^\star(t) = s_\theta\, \dot x(t) \qquad \Longrightarrow \qquad \varepsilon(t) = 0 \;\; \forall\, t$$

and tracking is exact. The interesting cases are the three ways a biological integrator departs from it, each of which produces a different growth law for $|\varepsilon|$ — which is what makes them separable from a single measured trace.

Gain. If the velocity-to-position conversion is miscalibrated by a factor k,

$$\frac{d\theta}{dt} = k\, \dot\theta^\star(t) \qquad \Longrightarrow \qquad \varepsilon(t) = (1-k)\left[\theta^\star(t) - \theta^\star(0)\right]$$

The error is proportional to the *displacement* from the start, not to the distance travelled, because integration is linear. In a bounded workspace displacement is capped by the arena, so this error is self-limiting: in our geometry the target reaches a median 1.16 arena units from the centre, and $|1-k|$ must exceed 0.52 before the target can leave a 0.6-unit fovea at all. A realistic miscalibration of a few percent is invisible. Gain is not what loses the target.

Leak. A real integrator is imperfect and relaxes toward its rest state with a time constant τ:

$$\frac{d\theta}{dt} = -\frac{\theta}{\tau} + \dot\theta^\star(t)$$

In the frequency domain this is a *high-pass* on target eccentricity, corner frequency $1/\tau$. Sustained excursions are under-represented, so the error saturates rather than growing without bound, and gaze is a shrunken, centre-biased copy of the truth. The counterintuitive consequence is that *slow* targets are lost first, because their motion is exactly the low-frequency content the leak removes. Measured: at $\tau = 2$ s a fast target is never lost within 20 s while a slow one goes at 5.9 s, and the slow case needs $\tau \geq 8$ s to survive. τ is the quantity the INTG pool exists to make long, and this is the curve that says how long is long enough.

Noise. If the integrated velocity carries a white perturbation of intensity σ,

$$\frac{d\theta}{dt} = \dot\theta^\star(t) + \sigma\, \xi(t), \qquad \langle \xi(t)\, \xi(t') \rangle = \delta(t - t')$$

the error is a random walk: $\mathrm{std}[\varepsilon(T)] = \sigma\sqrt{T}$ per axis, unbounded but with zero mean. It is the only one of the three that cannot be corrected by better calibration and the only one that averages away across trials, so it is invisible to any analysis that pools trials before measuring drift.

Three growth laws — bounded-linear, saturating, and square-root — over one shared quantity. A drift trace measured from the real circuit can therefore be *classified*, not merely reported, which is the point of setting the problem up this way. `prototype/dot_tracking/openloop.py` implements the gain and leak cases and measures the survival time `t_lose`, the first moment $|\varepsilon|$ exceeds the fovea; the noise case is stated here for

completeness but is not in the prototype, since a zero-mean drift is better measured against recorded data than against a simulation of itself.

3. The training dataset

3.1 Where it comes from

Task data is generated by `GNN_Main.py -o generate <config>`, which reaches `data_generate` -> `data_generate_task` (dispatch on `task.task_type`) -> `_generate_swim_integration_task`, all in `src/connectome_gnn/generators/graph_data_generator.py`.

The critical property: **generation is connectome-agnostic**. The generator writes `stimulus.zarr` of shape (trials, T, channels) and `target.zarr` of shape (trials, T, columns) — pure time series, with no notion of a neuron. The circuit appears only as a `circuit_provenance.json` written beside the splits, recording the circuit name, cell count and the SHA-256 of `J_effective`, so a dataset can later be checked against the circuit a checkpoint was trained on.

The consequence for planning: the dataset can be generated and inspected **before** the circuit registry entry, the connectome CSVs or the E/I assignment exist. That is the cheapest next step on this branch.

3.2 Parameters that define it

parameter	meaning
`n_trials_train` / `n_trials_test`	independent trials per split
`n_steps`, `dt`	samples per trial and the timestep, so trial duration is `n_steps * dt`
`swim_rate_hz`	mean Poisson rate of impulse onsets — the saccade rate
`swim_duration_s`	boxcar width of one impulse — the saccade duration
`forward_vel_mean` / `forward_vel_std`	amplitude distribution of the velocity drive
`target_kind`	which target is assembled; `scalar_xi` is the integrator
`xi_tau_s`	the leak; absent or <= 0 gives a perfect integrator
`seed`	stimulus RNG; fixed across baseline and comparison runs

`training.task_targets` then projects the on-disk superset down to the columns a given run trains on, so one dataset serves several sub-tasks.

4. Learning the controller

The task is **supervised, not reinforcement**, and it is worth being explicit about why, because the instinct to reach for RL here is strong and wrong. In open loop the correct output is known in closed form at every timestep — given the velocity stream, the target eye position is its integral — so the teacher is dense rather than sparse. And the environment does not react to the agent: the dot moves the same way whatever the eye does. With no feedback loop and no unknown to explore, this is sequence-to-sequence regression trained by backpropagation through time. RL would recover the same gradient information with far more variance and no compensating benefit.

Closed loop does introduce a loop, but not a reason to change method: the eye is `gaze = integral of command`, which is differentiable, so one simply backpropagates through it. RL earns its place only where the objective stops being differentiable — driving the MPM soft-body eye of `Plexus/prototype/eye` without backpropagating through the simulator, or choosing *when* to make a catch-up saccade, which is a discrete decision rather than a continuous command. Smooth pursuit is regression; saccade timing is a policy.

4.1 Two stages, in this order

Stage 1, in the prototype, with no biology in the way. Generate a corpus of trajectories with `trajectory.py`, and train a small unconstrained recurrent network — free $\hat{W}$, no Dale, no connectome — on exactly the task `openloop.py` measures. The point is not the model; it is the calibration. It answers how many trials

the task needs, what training horizon is required, and what integrator time constant is reachable at all when nothing anatomical constrains the solution. That number is the ceiling.

Stage 2, the synaptic solution. Replace the free recurrent matrix with the 285-cell sign-locked $\hat{W}$, keeping the same data, loss and curriculum, and keep the encoder and decoder small: the encoder maps the velocity signal onto the AF5 afferents, the decoder reads AMN/AIN. The gap between stage 1 and stage 2 is then interpretable as the cost of the anatomy, rather than as an unexplained training failure — which is exactly the comparison that cannot be made if the constrained model is trained first and alone.

4.2 Four things that will bite

1. **Credit assignment over the horizon.** Gradients through an integrator across thousands of steps. The heading-direction configs handle this with a step-count curriculum (`n_steps_schedule`, 100 -> 800) and a tail-weighted loss (`coeff_tail_loss`); expect to need both.
2. **`tau` is not identifiable from short trials.** On an 8 s trial a 20 s time constant and a perfect integrator are indistinguishable. If `tau` is the scientific quantity, the training horizon has to approach it, or the protocol needs an explicit hold-and-decay probe: drive, then stop, and watch the decay.
3. **Degeneracy.** Many recurrent matrices integrate. Sign-lock, Dale and spectral normalisation are what select among them, and the residual degeneracy is the same identifiability problem the zebrafish heading-direction work already documents.
4. **Signed position on non-negative rates.** AMN and AIN are motor pools with rates bounded below by zero, so eye position must be carried as a push-pull difference (lateral minus medial rectus) rather than by a free linear readout. That is what the horizontal pair physically is, and building it in is cheaper than hoping the decoder discovers it.

4.3 Stage 1, measured

The calibration has been run. A balanced corpus of 5160 centre-started trajectories — every combination of the four switches, 24 conditions, 8 s at 60 Hz, seeds disjoint across splits — and five learners with the same interface: linear encoder, swappable core, linear decoder. Scored on one shared 960-trial test set, mean |drift| in grid units:

controller	mean drift	never lost	tau_e
`perfect` (analytic ideal)	0.004	100 %	—
`ctrnn`	**0.007**	100 %	inf
`gru`	0.009	100 %	inf
`intlin` (fitted leaky integrator)	0.090	100 %	9.5 s
`gain`, k = 0.5	0.225	98 %	—
`leaky`, tau = 2 s	0.284	67 %	—
`mlp` (windowed, memoryless)	0.392	25 %	0.48 s
`rnn` (discrete tanh)	0.424	27 %	0.40 s

Two of the rows need comment here. Why the top one is a floor rather than a result, and where it comes from, is section 4.4.

A discrete tanh RNN cannot learn this, and that is an optimisation result, not a capacity one. It plateaus at 0.42 whether trained for 40 or 250 epochs, and identity-initialising the recurrent matrix does not rescue it. The gradient of an 8 s integral through a contracting discrete map vanishes. The continuous-time form fixes it because with `dt/tau` small the update is near-identity by construction. This matters for stage 2: had we used the discrete form as the stand-in, we would have concluded that the task was hard, when the difficulty was in the parameterisation.

The analytic controllers are lesions, not competitors. Their gain and time constant are not free choices to be tuned for a fair fight — fitted, they go to `k = 1` and `tau -> infinity`, which is just `perfect` again, because the task's ideal solution *is* a perfect integrator. `gain = 0.5` and `tau = 2 s` are deliberate defects, chosen so the drift is visible in the interface. The properly fitted member of that family is `intlin`, which learns

its own decay and lands at `tau = 9.5 s`. Read the analytic rows as "how bad is this specific defect", never as "how good is this method".

4.4 The ceiling, and where the integration lives

0.007 is the evaluation's floor, not the model's. `ctrnn` reaches 0.0074 against 0.0041 for exact analytic integration, and the gap is arithmetic — velocity is a central difference while the target is an Euler sum. It audits clean: disjoint seeds, the one-sample look-ahead worth 13 %, the time constant probed beyond any horizon trained on. Of four levers only training length moved it (0.0144 to 0.0074); nine times the recurrent parameters gave 0.0072 and train and validation MSE are identical, so the constraint was optimisation, not capacity and not data.

The integration is in the recurrent matrix, not the neurons. Learned time constants are 0.57 to 0.96 s, yet a hold-and-decay probe returns effectively infinite over 20 s: positive feedback through $\hat{W}$, initialised at exactly zero, cancels the leak and buys 27 times the neuronal time constant. That is the classical oculomotor-integrator result reached from the other direction — the optimiser could have grown τ_i instead and did not. Stage 2 should therefore measure the **feedback gain the connectome can support**, and perturb $\hat{W}$ to test it, because tuned positive feedback is fragile. (The memoryless MLP's 0.48 s decay constant is simply its own 0.5 s window.)

The controller rediscovered the pulse-step Watching the trained network drive the eye turns up something the training never asked for. Whenever the target reverses direction the **command** swings wildly while the **gaze** barely moves off the target: over the sharpest 3 % of turns the gap between command and gaze grows 3.9 times, from 0.56 to 2.18 degrees, while the gaze error grows only 1.33 times, 0.098 to 0.130. The obvious reading is that the motor output has gone unstable and the eye's own smoothing rescues it.

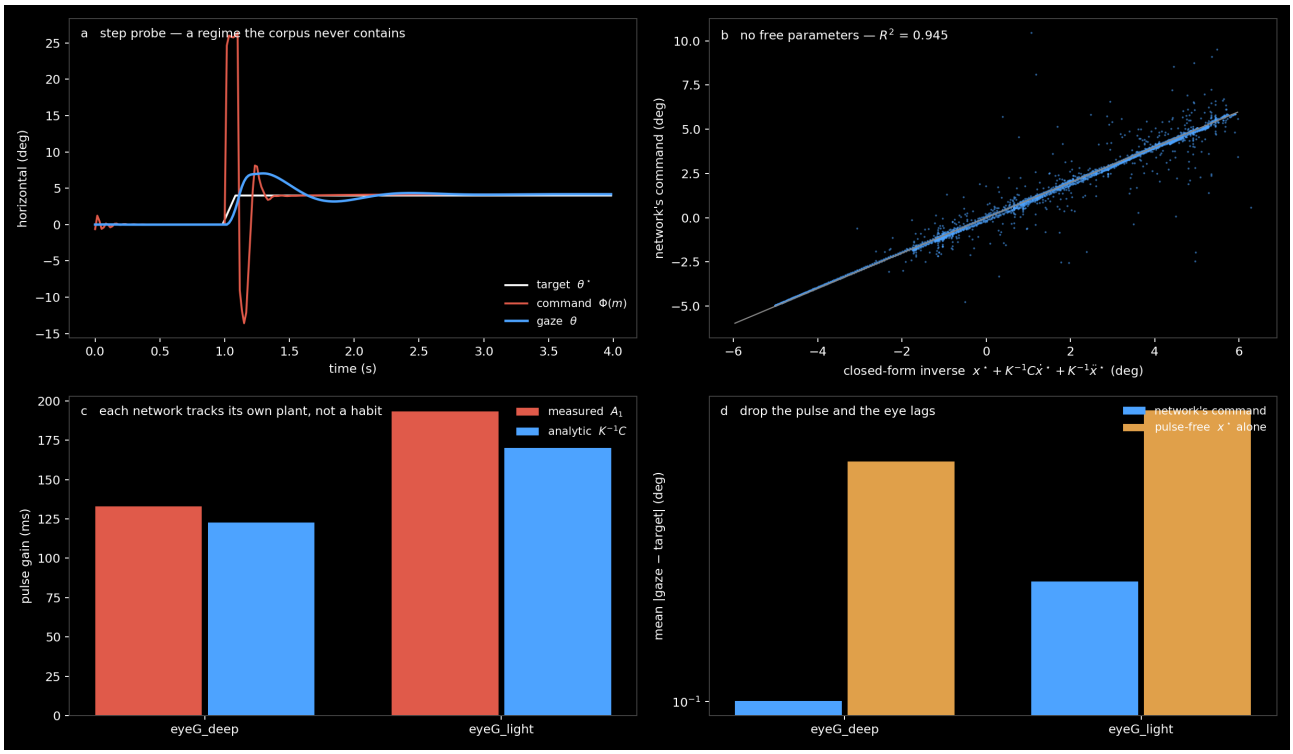
It is not instability. It is the eye's inverse, and the inverse has a closed form, so the claim is falsifiable. The eye of step 7 is

$$\ddot{\mathbf{x}} + C\,\dot{\mathbf{x}} + K\,\mathbf{x} \;=\; K\,\mathbf{x}_{\infty}$$

and rearranging for the command that would place the gaze exactly on a wanted trajectory $\mathbf{x}^{\star}(t)$ gives

$$\mathbf{x}^{\star}_{\infty} \;=\; \mathbf{x}^{\star} \;+\; K^{-1}C\,\dot{\mathbf{x}}^{\star} \;+\; K^{-1}\ddot{\mathbf{x}}^{\star}$$

Read it in plain terms: to hold the eye somewhere, ask for that place; to move it, ask for somewhere *further*, in proportion to how fast you want to go; and to start or stop the movement, add a term proportional to the acceleration. A step in $\mathbf{x}^{\star}$ makes the velocity term a pulse and the acceleration term a doublet, and what is left afterwards is the step. **That is Robinson's pulse-step**, falling out of the algebra rather than being designed in — the same pulse-step section 5.1 credits Robinson (1964, 1981) with, as the command that inverts the linear eye. Nothing in this model was told about it.



The controller learned the eye's inverse, not a tracking habit. (a) A step probe: the target steps 4° and holds. The corpus contains no steps — it is splines at three speeds — so this regime is new to the network. The command goes to +26, brakes to −14, then settles on the +4 step, while the gaze rises smoothly to the target: accelerate, brake, hold. The doublet is the $K^{-1}\ddot{x}^\star$ term and the offset that survives it is $x^\star$. **(b)** The network's command against the closed-form inverse above, evaluated with the eye's own K and C and **no free parameters**: $R^2 = 0.945$. Fitting A_0, A_1, A_2 freely reaches only 0.952, so the analytic matrices cost essentially nothing. **(c)** The velocity gain A_1 of each network against its own $K^{-1}C$. The two eyes differ by 46 % in measured pulse gain (133 against 193 ms) and each matches its own plant; a pulse that was a habit of the architecture or of the corpus would be the same in both. **(d)** Necessity: drive each eye with the pulse-free command $x^\star$ and the tracking error rises 5.9× on the deep eye (0.101 → 0.592°) and 3.5× on the light one. Regenerate with `python fig_pulse_step.py` in `prototype/dot_tracking`.

Why "learned" rather than "built in". Four things have to be true at once, and each is measured in the figure.

The shape is not memorised. The corpus is splines at three speeds and contains no steps at all; the step probe of panel (a) is a regime the network has never been in, and it produces the pulse-step there.

The coefficients are the plant's. Regressing the command freely on the target and its first two derivatives, $\mathbf{m}_{\text{cmd}} \approx A_0\mathbf{x}^\star + A_1\dot{\mathbf{x}}^\star + A_2\ddot{\mathbf{x}}^\star$, reaches $R^2 = 0.952$; *replacing* the fitted A_1, A_2 with the analytic $K^{-1}C$ and K^{-1} — no free parameters at all — still reaches 0.945. The network's velocity gain is 133 ms against the plant's 123. Note the regressor is the **target**, not the achieved gaze: regressing on the gaze would be circular, since the equation above makes that an identity for any command whatsoever.

It is this eye's inverse, not an eye's. The deep and light controllers were trained identically on the same corpus and differ only in the eye between them. Their pulse gains differ by 46 %, 133 against 193 ms, and each tracks its own $K^{-1}C$, 123 against 170. An architectural habit would not know which plant it was attached to.

The loss had no alternative. Driving the same eye with the pulse-free command — just $\mathbf{x}^\star$, no velocity or acceleration term — costs a factor of 5.9 in tracking error on the deep eye and 3.5 on the light one. The pulse is not decoration the optimiser could have skipped.

Two structural points make the result sharper. Nothing in the architecture supplies $\dot{\mathbf{x}}^\star$ or $\ddot{\mathbf{x}}^\star$: the readout is a static map of the rates, so the velocity term has to come from the input — which *is* the target velocity, scaled — while the position term must be integrated out of it and the acceleration term differentiated out of it, both

by the recurrent dynamics alone. And K and C appear nowhere in the loss; they are buffers inside a frozen eye the gradient passes *through*. The controller inferred them from the only thing it was ever shown, which is how far the gaze ended up from the target.

This is the same question as the paragraphs above — where does the computation live — asked of the trained controller rather than of its weights. There the answer was that the integration is in the recurrent matrix and not in the membranes. Here it is that the *inverse model* is in the recurrent matrix too, and that a network given only a velocity stream and a squared error at the far end of a plant will build one.

4.5 How the recurrent models are trained

Worth stating precisely, because two of the choices are what make the task learnable at all and a third is a known omission.

The state starts at $v_i = 0$ for every neuron on every trial, never learned and never carried between trials. That is why the corpus is centre-started: $v_i = 0$ has to *mean* "eye centred", so that the initial condition and the task agree. The loss is a plain mean squared error over **every timestep and both output channels** — dense supervision from t=0, not an endpoint loss.

The horizon grows during training, always as a prefix from t=0 rather than a sliding window:

epochs	horizon	
0-62	120 steps	2.0 s
63-124	240 steps	4.0 s
125-186	360 steps	6.0 s
187-249	480 steps	8.0 s

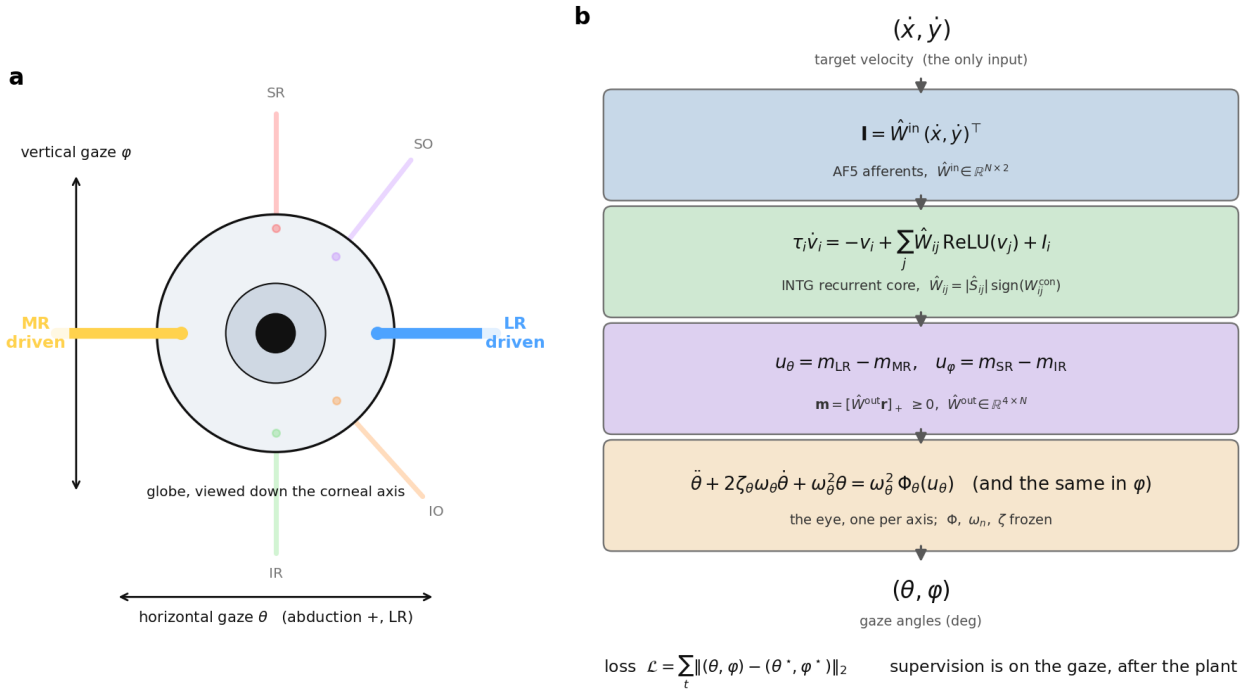
Each stage re-runs from $v_i = 0$ and truncates; there is no truncated BPTT and no carried state. Gradients flow through the whole unroll — 480 sequential steps at the last stage, no detach — with Adam at 2e-3 decayed by cosine to zero, gradient-norm clipping at 1.0, batch 128, so 29 updates per epoch and 7250 in total.

The curriculum is not a refinement, it is the reason this trains. At the full 8 s horizon from a cold start the gradient of the integral is exactly what defeated the discrete RNN; beginning at 2 s makes the 8 s case reachable. It is the same device as `n_steps_schedule` in the heading-direction configs.

What is missing is tail weighting. Early timesteps, where the integral is small and nearly free, currently count as much as late ones, so the loss under-rewards precisely the long-horizon accuracy the exercise is about. The heading-direction configs carry `coeff_tail_loss` for this reason and this prototype does not; adding it is the obvious next attempt at the residual gap to exact integration.

One reassurance about the horizon. The models are tested at the 480 steps they were trained on, but the hold-and-decay time-constant probe runs 1200 steps — 2.5 times anything seen in training — and the integration holds. Whatever the network learned, it is not a fit to the trial length.

5. The whole model in one notation



The eye, and the symbols of the model. (a) The globe seen down the corneal axis with the six extraocular muscles at their true insertions, drawn from `eye_anatomy.MUSCLES` — the same table the MPM uses to shape the straps, so this is the model's own geometry. Only the horizontal pair is innervated by this circuit: LR abducts (positive gaze), MR adducts. The four faint muscles exist in the eye but receive no command. The two axes the model uses are marked: horizontal gaze θ, positive in abduction, and vertical gaze φ. **(b)** Every symbol of the nine steps below, in the order the signal meets them: the target velocity $(\dot{x}, \dot{y})$, the input map $\hat{W}^{\text{in}}$ onto the AF5 afferents, the sign-locked recurrent core, the output map $\hat{W}^{\text{out}}$ onto four non-negative motor pools, the push-pull commands u_θ, u_φ, the frozen per-axis eye, and the loss on the gaze angles (θ, φ) — after the eye, not on the command. Regenerate with `python fit_plant.py` and `python fig_eye_schematic.py` in `Plexus/prototype/eye`.

Written in the formalism of `neurips.tex`, with the symbols laid out in the figure above, so the oculomotor circuit and the flyvis work can be read side by side. Every stage from the two numbers the circuit is given to the two angles it is scored on is written out below, because the intermediate quantities are where the modelling choices are, and they are invisible in any summary that jumps from "velocity in" to "position out".

symbol	dimension	meaning
$x(t)$, $y(t)$	arena units	target position in the world
$\dot{x}(t)$, $\dot{y}(t)$	units s^{-1}	target velocity — **the only input**
s_θ, s_φ	deg / unit	world-to-angle scale, one per axis
$\theta^\star$, $\varphi^\star$	deg	target eccentricity, horizontal and vertical
$v_i(t)$	—	membrane voltage of neuron i ($N = 285$)
τ_i	s	membrane time constant of neuron i
$r_i(t)$	—	firing rate, $r_i = \rho(v_i)$
$\hat{W} \in \mathbb{R}^{N\times N}$	—	recurrent weights, sign-locked to the connectome
$\hat{W}^{\mathrm{in}} \in \mathbb{R}^{N\times 2}$	—	velocity to afferents; rows zero outside AF5
$\hat{W}^{\mathrm{out}} \in \mathbb{R}^{4\times N}$	—	rates to muscles; columns zero outside the motor pools
$m_{\mathrm{LR}}, m_{\mathrm{MR}}, m_{\mathrm{SR}}, m_{\mathrm{IR}}$	≥ 0	motor-pool drives, one per muscle
u_θ, u_φ	$[-1, 1]$	push-pull commands, one per axis
Φ_θ, Φ_φ	deg	static nonlinearity of the eye, per axis
ω_n, ζ	rad s^{-1}, —	eye mechanics, per axis
$\theta(t)$, $\varphi(t)$	deg	**eye angles** — horizontal and vertical gaze

A hat marks a learned quantity. v is the membrane voltage throughout and never a velocity; the velocities are $\dot{x}$ and $\dot{y}$, and the eye's orientation is the pair (θ, φ), horizontal first.

Step 0 — the world sets the angles. The target moves in a bounded arena in units, and each axis is mapped to degrees by its own scale, because the eye's reachable travel is not the same horizontally and vertically (section 5.2):

$$\theta^\star(t) = s_\theta\, x(t), \qquad \varphi^\star(t) = s_\varphi\, y(t), \qquad \theta^\star(0) = \varphi^\star(0) = 0$$

Step 1 — what the circuit is given. Only the velocity, never the position. This is the open-loop premise of section 2.3 written as a vector:

$$\mathbf{u}(t) \;=\; \big(\dot{x}(t),\ \dot{y}(t)\big)^\top \in \mathbb{R}^2$$

Step 2 — the input map $\hat{W}^{\mathrm{in}}$. The drive enters only through the AF5 afferents $\mathcal{A}$ ($|\mathcal{A}| = 41$). That restriction is the whole content of "connectome-constrained input": a free input layer would let the optimiser inject velocity wherever it is most convenient, which is exactly the degree of freedom the anatomy is supposed to remove:

$$\mathbf{I}(t) \;=\; \hat{W}^{\mathrm{in}}\,\mathbf{u}(t), \qquad \hat{W}^{\mathrm{in}}_{i,:} = \mathbf{0} \quad \forall\, i \notin \mathcal{A}, \qquad \mathcal{A} = \mathcal{A}^L_{\mathrm{AF5}} \cup \mathcal{A}^R_{\mathrm{AF5}}$$

so $\hat{W}^{\mathrm{in}}$ is 285×2 with $41 \times 2 = 82$ free entries out of 570. Componentwise, neuron i receives $I_i = \hat{W}^{\mathrm{in}}_{i1}\dot{x} + \hat{W}^{\mathrm{in}}_{i2}\dot{y}$, and the direction tuning of section 1.3 — still a CLAIM — is what says whether that row should be free or fixed to a preferred direction.

Step 3 — the circuit. The $N = 285$ neurons obey the same rate equation as Eq. (1) of `neurips.tex`:

$$\tau_i \frac{dv_i(t)}{dt} \;=\; -\,v_i(t) \;+\; V_i^{\mathrm{rest}} \;+\sum_{j\in\mathcal{N}_i} \hat{W}_{ij}\, r_j(t) \;+\; I_i(t) \;+\; \sigma\,\xi_i(t), \qquad r_j = \rho(v_j)$$

with $\mathcal{N}_i$ the presynaptic partners of i and ρ the rate nonlinearity — ReLU in the constrained model, tanh in the prototype of section 4.3.

Step 4 — the sign lock. Only the magnitudes are learned; the signs are the measured Dale assignment of section 1.4, plotted in panels (c) and (d) of Figure 1, so $\hat{W}$ never leaves the connectome's sign pattern:

$$\hat{W}_{ij} \;=\; \big|\hat{S}_{ij}\big|\,\mathrm{sign}\big(W^{\mathrm{con}}_{ij}\big), \qquad \mathrm{sign}\big(W^{\mathrm{con}}_{ij}\big) = \begin{cases} -1, & j \in \mathcal{I} \quad \text{(INTG contra, 52 cells)} \\ +1, & \text{otherwise} \quad \text{(233 cells)} \end{cases}$$

Step 5 — the output map $\hat{W}^{\mathrm{out}}$. The motor pools are read as non-negative drives, one per muscle, ordered LR, MR, SR, IR:

$$\mathbf{m}(t) \;=\; \big[\,\hat{W}^{\mathrm{out}}\mathbf{r}(t)\,\big]_+ \;=\; \big(m_{\mathrm{LR}},\, m_{\mathrm{MR}},\, m_{\mathrm{SR}},\, m_{\mathrm{IR}}\big)^{\top}, \qquad \hat{W}^{\mathrm{out}}_{:,i} = \mathbf{0} \quad \forall\, i \notin \mathcal{M}$$

with $[\cdot]_+$ a rectifier (softplus in the prototype) enforcing $m_\bullet \geq 0$, and $\mathcal{M}$ the motor pool. In the selected 285 cells $\mathcal{M}$ is AMN (92 cells, row LR) and AIN (35 cells, row MR) only, so $\hat{W}^{\mathrm{out}}$ is 4×285 with two live rows and 127 live columns. **The vertical pair has no anatomical pool here**: SR and IR are innervated by OMN, which section 1.5 leaves out of the selection. The two-axis form is written because the eye and the prototype are two-axis; the constrained circuit as currently scoped commands θ alone, and reaching φ means adding OMN to `circuit.cell_types`.

Step 6 — push-pull. Each axis is the difference of an antagonist pair, so a signed angle is carried on strictly non-negative rates by construction rather than by a free linear readout that might or might not discover it (hazard 4 of section 4.2):

$$u_\theta(t) = m_{\mathrm{LR}}(t) - m_{\mathrm{MR}}(t), \qquad u_\varphi(t) = m_{\mathrm{SR}}(t) - m_{\mathrm{IR}}(t)$$

Step 7 — the eye, per axis. Each command drives a Hammerstein eye: a monotone static nonlinearity followed by second-order mechanics, with Φ, ω_n and ζ identified in section 5.1 and **frozen** — steps 6 and 7 together are what this note calls the **lateral-horizontal Hammerstein model**, and section 5.3 says what has to replace it — registered as buffers, not parameters, because the body is measured and letting the optimiser retune it would defeat the point:

$$\Phi_\theta(u_\theta) = a_\theta\, u_\theta + b_\theta\, u_\theta^2, \qquad \ddot{\theta} + 2\zeta_\theta\omega_\theta\,\dot{\theta} + \omega_\theta^2\,\theta \;=\; \omega_\theta^2\,\Phi_\theta\big(u_\theta(t)\big)$$

$$\Phi_\varphi(u_\varphi) = a_\varphi\, u_\varphi + b_\varphi\, u_\varphi^2, \qquad \ddot{\varphi} + 2\zeta_\varphi\omega_\varphi\,\dot{\varphi} + \omega_\varphi^2\,\varphi \;=\; \omega_\varphi^2\,\Phi_\varphi\big(u_\varphi(t)\big)$$

The two axes have their own (Φ, ω_n, ζ), fitted from the LR/MR and the SR/IR probes separately; on eye C they differ by more than a factor of one and a half in travel, which is why $s_\theta \neq s_\varphi$ in step 0.

Step 8 — the objective. Supervision is on the eye angles *after* the eye, never on the command, in the same summed form as Eq. (3) of `neurips.tex`:

$$\mathcal{L}_{\mathrm{pred}} = \sum_t \Big\| \big(\theta(t), \varphi(t)\big) - \big(\theta^\star(t), \varphi^\star(t)\big) \Big\|_2$$

Putting the loss after the eye is what makes the network learn the eye's inverse rather than a velocity command: the gradient reaches $\hat{W}, \hat{W}^{\mathrm{in}}, \hat{W}^{\mathrm{out}}$ through Φ and through the second-order mechanics, so a different eye trains a different controller. That is why section 5.2 lists one trained network per eye rather than one network tested on five.

Step 9 — discretisation and initial condition. Forward Euler at $\Delta t = 1/60$ s, with the network state started at rest on every trial:

$$v_i(t+\Delta t) = v_i(t) + \frac{\Delta t}{\tau_i}\Big(- v_i(t) + \textstyle\sum_j \hat{W}_{ij} r_j(t) + I_i(t)\Big), \qquad v_i(0) = 0, \quad \theta(0) = \varphi(0) = 0$$

$v_i(0) = 0$ has to *mean* "eye centred", which is why every trajectory in the corpus starts at the centre (section 4.5). Nothing is carried between trials.

Two differences from the flyvis setting are worth naming. There the supervision is on dv_i/dt of *every* neuron, because the simulator provides the full state; here it is on two scalars at the far end of an eye, so the circuit is constrained only through its behavioural consequence. And there $\hat{W}$ is what is being recovered from activity, whereas here $\hat{W}$ is fixed in sign by the connectome and only its magnitudes, $\hat{W}^{\text{in}}$ and $\hat{W}^{\text{out}}$ are free. The oculomotor problem is thus the better-posed of the two: fewer unknowns, but a much narrower observation.

What the prototype of section 4.3 instantiates. The same nine steps with the connectome removed, which is the point of stage 1: $N = 64$ free units in place of the 285 cells, $\rho = \tanh$, $\hat{W}$ dense and initialised at zero with no sign lock, $\hat{W}^{\text{in}}$ a dense 64×2 layer instead of 82 entries on AF5 rows, $\hat{W}^{\text{out}}$ a dense 4×64 layer with all four muscle rows live. Steps 6 to 9 — push-pull, the frozen per-axis eye, the loss on (θ, φ), the Euler step from $v = 0$ — are identical, so the gap between the two is attributable to the anatomy and to nothing else.

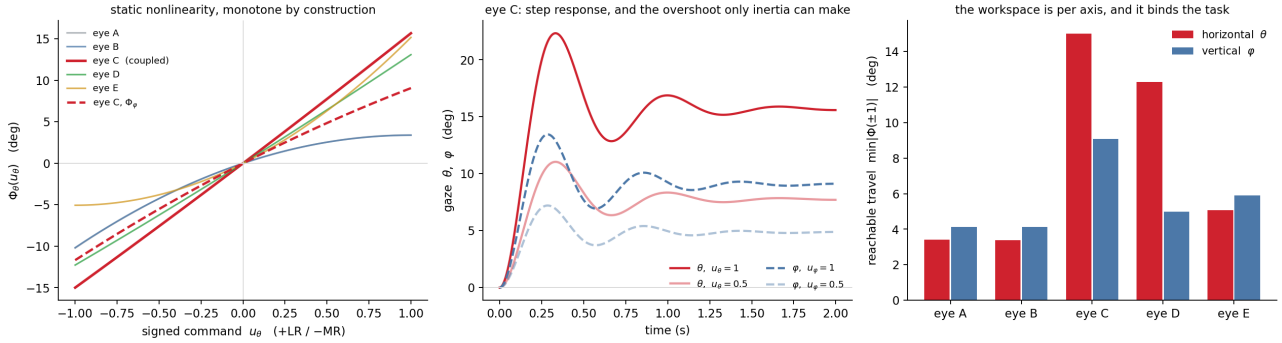
5.1 The eye model

The circuit's output is muscle drive, not eye position, so the mechanics has to be in the loop — and the MPM eye cannot be: its grid scatter is in-place, so it is not differentiable, and one trial costs more than all 7250 gradient updates of section 4.5. It is replaced by the reduced model of step 7, fitted by `fit_plant.py` from the probe runs in `Plexus/prototype/eye/archive/`. Steps 6 and 7 together are the **lateral-horizontal Hammerstein model**: two antagonist pairs, two scalar commands, two independent single-input eyes.

Putting the whole nonlinearity in a memoryless block and leaving the dynamics linear is the **Hammerstein** structure — the standard first move of block-oriented nonlinear system identification (Narendra and Gallman 1966; Bai 1998; Giri and Bai 2010), which dominates practice because it keeps the linear factor a transfer function, with every tool of linear systems theory intact, while the nonlinear factor stays a curve fitted pointwise. For an eye the split is anatomical rather than convenient: the nonlinearity is muscle and memoryless, the dynamics are globe and orbital tissue and near-linear at these rotations. It is also the classical oculomotor model — Robinson (1964, 1981), with later work arguing the real eye carries more time scales than second order allows (Sklavos, Porrill, Kaneko and Dean, 2005).

The factorisation earns its keep here — across the five eyes the mechanics barely move, $\omega_n = 9.5$ to 11.1 rad s^{-1} and $\zeta = 0.22$ to 0.32, while the static gain ranges over 14 to 23 degrees of travel. The configurations change Φ, not the ODE, so one set of dynamics covers the sweep. At $\zeta \approx 0.3$ the modelled eye is underdamped and rings at about 1.5 Hz; real eyes are overdamped, so that is a property of the MPM configuration rather than of a fish.

Φ and the mechanics are fitted **jointly**, against whole trajectories. The alternative — read the plateaus for Φ, then fit the transient — needs settled plateaus, and this archive has none, for the reason given below.



The identified eye models. All three panels are drawn from the stored fit in `plant.npz` / `plant_v.npz`, in the symbols of section 5. **(a)** The static nonlinearity $\Phi_\theta(u_\theta)$ of all five eyes, and Φ_φ of eye C, the one the controller is coupled to. Every curve is monotone by construction; the curvature is the genuine asymmetry between abduction and adduction. **(b)** Step response of eye C on both axes at commands 0.5 and 1: the overshoot and the ringing are what only the inertial term can produce, and they are the reason the second-order eye beats the first. **(c)** Reachable travel per axis, $\min|\Phi(\pm 1)|$ — the quantity that decides whether a tracking task is expressible at all, and the reason the world is scaled anisotropically. Regenerate with `python fig_plant_summary.py` in `Plexus/prototype/eye`.

Second order wins on every variant, by a factor of 3. Against the first-order alternative $\tau\dot{\theta} + \theta = \Phi_\theta(u_\theta)$ — viscosity and an elastic restoring force, no globe inertia — the RMS error in degrees is:

variant	order 1	order 2	ratio
B `baseline_fixmat`	1.22	**0.41**	3.0x
A `c_a`	1.20	**0.42**	2.9x
C `p3a_length`	2.58	**0.62**	4.2x
D `p3b_pulley`	2.24	**0.77**	2.9x
E `p3c_drive`	6.77	6.34	1.1x

This is not a matter of degrees of freedom. A first-order system is monotone towards its target and *cannot overshoot*, while the MPM eye overshoots and rings — panel (b). Only the inertial term can produce that. The one variant where the two orders are indistinguishable, `p3c_drive`, is also the one neither fits at 6 degrees of error, which is a signal about that run rather than about the model order. And the fit has to be **per variant**: pooled across all five it gives RMS 5.05 deg and a static curve with plateaus at +16, +13 and +3 degrees for the same command; fitted separately, 0.41 to 0.77 deg.

What the eye does to a moving target In the prototype interface the red trace is $\Phi(u)$ — where the eye *would* settle if the command froze — and the blue trace is where the eye actually is. At the speeds the task uses the two sit on top of each other, which looks as though the eye were doing nothing at all. It is not, and the reason is worth stating carefully, because it is the difference between an eye that distorts the command and one that merely postpones it.

Everything follows from the transfer function of step 7, the ratio of gaze to static command in the frequency domain:

$$H(s) \;=\; \frac{\Theta(s)}{\Phi(s)} \;=\; \frac{\omega_n^2}{s^2 + 2\zeta\omega_n s + \omega_n^2}$$

It does not shrink the target. $H(0) = 1$ exactly, by construction. Hold any command and the eye ends up precisely at $\Phi(u)$ — no steady-state error, no gain to calibrate away. This is the property that makes red and blue *equal* rather than merely proportional.

It delays it. Expanding at low frequency,

$$H(s) \;=\; 1 - \frac{2\zeta}{\omega_n}s + O(s^2) \;\approx\; e^{-\Delta s}, \qquad \Delta = \frac{2\zeta}{\omega_n}$$

which is a **pure time delay** to first order. Slow in, same thing out, Δ later. For eye C that is 54 ms horizontally ($\zeta = 0.263$, $\omega_n = 9.76$ rad s^{-1}) and 40 ms vertically (0.225, 11.24) — about three frames at 60 Hz, which is why red sits on blue.

Measured on the simulator itself, by driving one axis with a small sinusoid and reading the phase, against the closed form above:

target	lag, horizontal	closed form	gain |blue|/|red|
0.10 Hz	54.0 ms	54.0	1.003
0.20 Hz	54.6 ms	54.6	1.014
0.35 Hz	56.3 ms	56.4	1.043
0.60 Hz	61.6 ms	62.1	1.135
1.00 Hz	81.1 ms	83.3	1.445
1.50 Hz	143.9 ms	152.5	1.935

Read it left to right. Below about 0.3 Hz the eye is a pure 54 ms delay at unity gain, and red and blue are the same curve shifted by three frames. Above that the delay grows and the gain **rises** — rises, not falls, because $\zeta \approx 0.26$ is underdamped, so the response peaks near $\omega_n/2\pi = 1.55$ Hz at roughly

$$|H|_{\max} \approx \frac{1}{2\zeta} = 1.9$$

At 1.5 Hz the eye overshoots its own command by a factor of two. That amplification, not any lag, is what the small loops at direction reversals in the interface are: the command turns, the eye carries past it, and the two traces separate for as long as the ringing lasts.

Two consequences. For reading the interface, red on blue means the eye has caught up with its command, not that the eye is trivial — the static map is still doing a factor of 15.3 degrees per unit of command, and the plot only looks tidy because it is drawn *after* that map. For the task, the useful number is 0.3 Hz: below it the circuit can ignore the mechanics and solve a pure inversion of Φ, above it the mechanics are part of the problem and the network has to learn a phase lead as well.

A correction. An earlier note recorded the command leading the gaze by 0 ms at slow and middle speeds, from a cross-correlation. That was an artefact: the correlation peak of two smooth slow traces is flat across many frames, so the frame-quantised estimate collapsed to zero. Measured by single-frequency phase, the lag is 54 ms at every speed the task uses, and agrees with the closed form to a tenth of a millisecond.

5.2 The five eyes

The archive is a sweep over mechanical configurations, not repeats of one globe. They are labelled A-E in the order they were made, and the folders `Plexus/prototype/eye/archive/eye_A..E/` collect each one's probe runs, movies and curves.

eye	variant	what changed	horizontal travel
A	`eye_probe_c_a`	soft tissue: sclera Young's 300, tendon 9, bone cap 40 — the softest globe	-10.1 / +3.4 deg
B	`eye_probe_baseline_-fixmat`	same geometry, materials stiffened to 420 / 45 / 130. The reference eye	-9.7 / +3.2 deg
C	`eye_p3a_length`	B with the muscle gap widened 0.020 to 0.042 and strap fraction 0.55 to 0.95 — more contractile length, the largest travel	-14.4 / +15.1 deg
D	`eye_p3b_pulley`	C plus a `muscle_sleeve` constraint (k 2500, c 30, free 0.70-0.88): a connective-tissue pulley holding the strap to the globe	-11.4 / +12.2 deg
E	`eye_p3c_drive`	D with the drive amplitude raised 60 to 67	-4.5 / +13.5 deg

A and B are strongly asymmetric — barely 3 degrees of abduction against 10 of adduction — so neither can serve a tracking task whatever its fit quality. C is the one to couple to: nearly symmetric and the widest workspace.

A correction: there is no steady state in this archive The first version of this fit used a free cubic and produced a static curve whose slope at the origin was **negative** — pulling the lateral rectus would rotate the eye medially. That is not a finding about the eye, it is an artefact, and it invalidated everything downstream of it.

The cause was mundane. `static_curve` reads plateaus as steady states, but at $\omega_n = 10$ rad s^{-1} and $\zeta = 0.31$ the settling time $4/(\zeta\omega_n)$ is 1.28 s, while the holds in these runs last **0.19 s, 1.27 s and 0.24 s**. Not one hold in the archive has settled; the two short ones are still moving at 70 deg/s when sampled. What was called a static curve was measured entirely from transients, and the post-step return — the eye sailing back through zero after a full step was released — was being read as "small positive command gives negative gaze".

Φ is now **monotone by construction**: in the quadratic of step 7 the linear coefficient is parameterised as $a = e^p > 0$ and the quadratic one as $b = \frac{a}{2}\tanh q$, which forces $\Phi'_\theta(u_\theta) = a + 2b\,u_\theta > 0$ across the whole command range $u_\theta \in [-1, 1]$, and likewise in φ. The fit therefore cannot return a physically impossible curve whatever the data does, while the quadratic term still carries the genuine asymmetry between abduction and adduction. Every variant now has a positive slope at the origin and zero non-monotone fraction. The residuals rise a little — 0.41 to 0.72 deg on B — and that increase is the honest cost of refusing to fit transients.

The identification still needs re-running with **holds longer than 1.3 s at intermediate levels** (0.2 / 0.4 / 0.6 / 0.8 in both directions). Each protocol steps straight to full activation, so there are only two plateaus per variant, at command -1 and $+1$, and the curve between them is extrapolation constrained to be monotone rather than measurement. The dynamics are well determined, because those come from the transients; the static gain is not.

A caveat on reproducibility. The `archive/t*_probe_*/curves.npz` behind eyes A-E have since been deleted, so `fit_plant.py` can no longer be re-run on them and the figure above is drawn from the stored coefficients instead. The fit itself survives — `plant.npz` and `plant_v.npz` carry Φ, ω_n and ζ per variant, which is what the trained controllers use — but the staircase re-probe will have to regenerate the runs from the specs.

Coupling the circuit to the eye: what the geometry forces Driving the eye is not a matter of appending mechanics to the readout. Two constraints come from the mechanics and neither is negotiable by training.

Four pools, two antagonist pairs. The readout emits four non-negative motor drives and each axis is commanded by a difference, which is step 6 above. A single command cannot serve both axes: they have

different static curves and different mechanics, fitted separately from the LR/MR and the SR/IR probes. On eye C they differ by more than a factor of one and a half in travel.

The workspace is per axis, and it is the binding constraint. Reachable travel, in degrees, plotted in panel (c):

eye	horizontal	vertical	usable world
A	3.4	4.1	3.6 deg/unit
B	3.4	4.1	3.6
C	**15.0**	**9.1**	9.5
D	12.3	5.0	5.3
E	5.1	5.9	5.3

An isotropic task can only be scaled to the *smaller* of the two. Scaling it to the horizontal instead — which an implementation does by default, because the horizontal pair is the one everybody models — puts the vertical target outside the eye's travel 40 % of the time on eye C and 60 % on eye D. The eye then cannot look where the target is, and the resulting error is indistinguishable, in any plot, from a controller that has failed to learn. The prototype avoids it by scaling each axis to its own reach, which is the $s_\theta \neq s_\varphi$ of step 0.

That is the transferable point for the connectome model. Before any training result is interpreted, the reachable range of each output axis has to be compared against the eccentricity the task demands of it. A circuit whose motor pools cannot produce the required rotation will look exactly like a circuit that cannot compute — and only the first of those is fixed by anatomy rather than by optimisation.

Co-contraction is a second input, not a second value of the first Step 6 writes each axis as one signed scalar, which assumes reciprocal innervation: one muscle pulls, the other releases. If instead both pull — the lateral rectus leading and the medial rectus resisting — the difference is unchanged but the **sum** is not, and the sum is not a position command at all. Co-contraction raises the stiffness and the damping of the eye, so it moves ω_n and ζ rather than Φ.

A single-input Hammerstein cannot express that. Capturing it needs $\Phi(m_{\mathrm{LR}}, m_{\mathrm{MR}})$ together with $\omega_n(m_{\mathrm{LR}} + m_{\mathrm{MR}})$, identified from a probe that sweeps the sum as well as the difference. This is not academic for the present model: the motor readout already emits two independent non-negative drives per axis, so the network is free to co-contract, and the eye will ignore it — the capacity is there and its effect is silently discarded.

5.3 Characterising the MPM eye, from now on

The lateral-horizontal Hammerstein model of steps 6 and 7 assumes the eye is two independent axes driven by two signed scalars. The first properly settled measurements say it is not. On eye F the lateral rectus produces 3.86 degrees of torsion at full drive against 6.17 of horizontal — 63 % — and the inferior oblique produces 11.08 degrees of torsion, larger than any horizontal action in the eye. A model with no torsion coordinate cannot represent that, and a model with two commands cannot be told about it. This section is the replacement, and the protocol that identifies it.

The aim is a procedure rather than a model of one eye. Eye F will be replaced, eye G is coming, and the point of writing the protocol down is that neither should require re-deriving anything here.

The shape of the model, coarsest first The eye takes six muscle drives and returns three angles:

$$\mathbf{m}(t) \in [0,1]^6 \;\longrightarrow\; \mathbf{x}(t) = \big(\theta(t),\ \varphi(t),\ \psi(t)\big) \in \mathbb{R}^3$$

horizontal, vertical and torsion, in degrees. The drives are one-sided because muscles pull and do not push, which is why they are not the signed u_θ, u_φ of step 6.

The single structural assumption is that the eye is a **static map followed by linear mechanics**. Where the eye eventually comes to rest is a nonlinear function of the drives; how it gets there is linear:

$$\mathbf{x}_\infty = g(\mathbf{m}), \qquad \ddot{\mathbf{x}} + C\,\dot{\mathbf{x}} + K\,\mathbf{x} = K\,\mathbf{x}_\infty$$

In plain terms: hold the muscles at some fixed activation and the eye settles somewhere — that is g. Change the activation and the globe swings to the new resting place with an overshoot and a ring — that is C and K. It is the same factorisation as section 5.1, widened from one input and one angle to six inputs and three angles, and it is what makes the whole thing measurable: g is memoryless, so **it can be measured entirely from holds**, with no differential equation anywhere in the fit.

That is not our idea. It is the block-oriented structure whose provenance section 5.1 gives — Hammerstein's cascade, the identification literature from Narendra and Gallman (1966) through Bai (1998) to Giri and Bai (2010) — and for an eye it is also the classical description, Robinson (1964, 1981). What is new here is only that the blocks are multi-input and multi-output.

The static map, specifically Six inputs is too many to write in closed form and too few for a blind neural network to fit from an affordable number of simulations. The standard way out is to decompose the function by interaction order — the **functional ANOVA** decomposition, due to Hoeffding (1948) and made a practical tool by Sobol' (1993), which is also the basis of the additive models of Hastie and Tibshirani (1986):

$$g(\mathbf{m}) = \underbrace{\sum_{i=1}^{6} \phi_i(m_i)}_{\text{one muscle at a time}} + \underbrace{\sum_{i<j} \phi_{ij}(m_i, m_j)}_{\text{pairs}} + \underbrace{\eta(\mathbf{m})}_{\text{everything else}}, \qquad \phi_i : [0,1] \to \mathbb{R}^3$$

Read left to right this says: most of what the eye does is each muscle acting on its own, some of it is two muscles fighting or helping each other, and whatever remains is a small correction. The first term is six curves, each measured by driving one muscle alone. The second is fifteen surfaces. The third is a small neural network regularised towards zero, present so that the model can absorb what the first two miss rather than pretend it does not exist.

The reason this matters practically is sample size. A neural network on the six-dimensional cube would need thousands of simulated holds; the marginals need thirty, and the pairs are decided one at a time. The decomposition is what turns an unaffordable experiment into a two-hour one.

Nothing in g is constrained to be monotone. The negative-slope disaster of section 5.2 was caused by fitting transients as though they were plateaus, not by the fitting method, and with genuinely settled holds the constraint is unnecessary. It also turns out to have been actively harmful: the monotone parameterisation caps the curvature at $|b| \le a/2$, and eye F's horizontal recti need $b/a = 0.65$, so enforcing it doubles the residual from 0.54 to 0.95 degrees. Monotonicity is now **checked and reported**, not imposed.

The mechanics, specifically C and K are three-by-three, so the model can express one axis dragging on another, which the two independent second-order eye models of section 5.1 cannot. They are parameterised through their Cholesky factors,

$$K = L_K L_K^\top + \varepsilon I, \qquad C = L_C L_C^\top$$

which makes them positive definite and the eye therefore **stable by construction**, for any values the optimiser reaches. This replaces the per-eye supervision of ω_n and ζ: instead of assuming two numbers, the eigenvalues of the fitted pair are reported, and if the eye turns out to need more time scales than two — which Sklavos, Porrill, Kaneko and Dean (2005) argue real eyes do — that shows up as a poor fit rather than as a silently wrong assumption.

One optional block covers co-contraction, the failure of section 5.2 that no single-input model can express. Pulling two antagonists together leaves the difference unchanged and raises the stiffness, so the mechanics are allowed to depend on the total drive $s = \mathbf{1}^\top \mathbf{m}$:

$$K(s) = K_0\,(1 + \kappa_K s), \qquad C(s) = C_0\,(1 + \kappa_C s)$$

This is a linear-parameter-varying eye, and it is included only if the measurement below says it is needed.

The protocol Five stages, in `Plexus/prototype/eye/PROTOCOL_eye_characterisation.md`. Two choices in it are worth stating here because both were paid for in lost data.

Stage 0, the gate — six runs. Each muscle alone at full drive. This returns the reachable span per axis and the settling time. The controller needs 15 degrees horizontal and 10 vertical; an eye that cannot reach that cannot do the task, and characterising it is wasted compute. The horizontal figure was 25 degrees until eye G measured what traced anatomy actually delivers — 15.9 degrees on the cardinal synergies, 17.5 with the inferior oblique recruited nasally — against 7.9 on eye F, with the drive and globe-size levers both exhausted. A gate no eye can pass is not a gate, so the requirement was lowered to meet the anatomy. The cost is stated rather than buried: the task now asks for about half the horizontal excursion, so errors measured under it are not comparable with the eye C numbers earlier in this section. Eye F fails — 7.9 degrees horizontal from single-muscle extremes, 10.8 even allowing every muscle to co-activate helpfully, against 25 — so it is the worked example of the gate doing its job. Stage 0 also sets the hold length for everything that follows, $T_{\text{hold}} = \max(2\text{ s}, 1.5 \times \text{settling})$, derived per eye. A fixed constant is exactly how eyes A to E came to be fitted entirely from transients.

Stage 1, the marginals — thirty holds. Each muscle alone at $m_i \in \{0.10, 0.25, 0.50, 0.75, 1.00\}$. Five levels weighted to the low end, because eye F's nonlinearity is strongly convex — the lateral rectus gains 6.9 times more per unit drive at $m = 1$ than at $m = 0$ — so the shape lives near the origin, which is also where a tracking controller spends its time.

Stage 2, which pairs matter — fifteen holds, then nine per pair that does. Stage 1 gives an additive prediction for any combination. Drive each of the fifteen pairs at $m_i = m_j = 0.5$ and compare; a pair whose residual exceeds 0.2 degrees, four times the settling tolerance, gets a three-by-three grid, and a pair below it gets nothing more. This is interaction screening in the sense of classical design of experiments (Box, Hunter and Hunter), and it is where the saving is: a pair that does not interact becomes a recorded measurement instead of an untested assumption, at a cost of one run.

Stage 3, the mechanics — about twenty-five trajectories. Steps from rest in many directions, single-muscle frequency sweeps to pin the damping, and three matched pairs that reach the same final angle with different total drive — the last of these being the measurement that decides whether $K(s)$ and $C(s)$ are needed at all.

Stage 4, fit and select. g from the holds alone, by ordinary regression; C and K from the trajectories with g frozen; then a short joint refinement of both against the trajectories, which is what rescued the eye C fit. Then a nested comparison on held-out runs: marginals against marginals-plus-pairs against plus-network, diagonal against full C, K, constant against drive-dependent. **This selection step is what makes the procedure general.** Every eye runs the same comparison and the data chooses how much structure that eye needs, so eye G requires re-running the selection and not rewriting the model.

What it costs About 112 runs against the 200 a blind low-discrepancy sweep of the cube would take, the saving coming entirely from stage 2. Measured on the target hardware, the whole protocol is **two hours on eight L4 GPUs**, which puts a full re-characterisation within a single working session and means an eye can be changed and re-measured in the same day.

What it changes upstream Two things in section 5, and they are simplifications rather than complications.

Step 6 disappears. There is no push-pull and no u_θ, u_φ; the readout emits six non-negative drives straight to the eye,

$$\mathbf{m}(t) = \left[\hat{W}^{\text{out}}\mathbf{r}(t)\right]_+, \qquad \hat{W}^{\text{out}} \in \mathbb{R}^{6\times N}$$

Step 8 gains one term. Six drives against two supervised angles leaves a four-dimensional set of muscle patterns producing the same gaze, and the network will wander in it arbitrarily. Penalising torsion pins it down:

$$\mathcal{L} = \sum_t \Big\| (\theta, \varphi) - (\theta^\star, \varphi^\star) \Big\|_2 \; + \; \lambda_\psi \sum_t \psi(t)^2$$

This is not an arbitrary regulariser. Real eyes resolve the same redundancy by holding torsion to a fixed function of gaze direction — Donders' law, and its sharper form Listing's law — and the term above is its simplest version. The three-dimensional treatment of eye rotations that makes this precise is Tweed and Vilis (1990) and Haslwanter (1995); adopting the full form later costs nothing extra, because the torsion coordinate is now in the model.

When the eye is good enough The same numbers for every eye, reported by the fit: the fraction of holds that settled, held-out RMS per axis in degrees, the fraction of the command cube where the fitted map is monotone in each muscle's own dominant axis, the reachable span per axis against the 15 and 10 degrees the task needs, the eigenvalues of (C, K), and the size of the interaction and residual terms relative to the marginals. An eye is usable when the span passes, the settled fraction is near one, and the held-out error is small against the precision the tracking task is scored at. Those are the criteria eye F fails on the first, and they are the criteria eye G will be judged by without anything in this section changing.

6. Progress, 11 August 2026

Opened the branch `feat/oculomotor` and put the zebrafish configs back under version control — 254 of them had been absent from every worktree since the `config/zebrafish` bind-mount was commented out of `devcontainer.json`, which left no figure script able to resolve a run config. Traced how the 917-cell heading-direction circuit was actually built (the colleagues' pickle, not the neuPrint query the filenames suggest) and wrote that up as `HOWTO_add_oculomotor_circuit.md`, since the same five inputs are needed again here. Read the new `Oculomotor_sortedData_081126.pkl`: 2949 cells, 42 types, 5 keys, no per-cell ordering variable; confirmed its matrix orientation empirically rather than assuming it. Selected the sub-circuit in two steps — first the 244-cell INTG/AMN/AIN core, then 285 cells once AF5 was added, which gave the pool the afferent stage it had been missing. Introduced `CellTypeSpec` so each type's pool, Dale sign, L/R split and role live in the config where they can be reviewed, rather than in hardcoded prefix tuples inside the loader. Rendered the two-panel connectivity figure above, and wrote this note.

Nothing has been trained, and no connectome CSVs exist yet. The five decisions below are what stand between this document and a first run.

7. What has to be decided next

1. **The AF5 combination rule** (§1.3) — AND, OR or XOR. Blocks the input model.
2. **Whether `OMN` joins the pool** (§1.5) — decides whether `AIN` -> medial rectus is one synapse or two.
3. **Whether `Burst_*` joins the pool** (§1.6) — decides whether the saccadic regime has an anatomical input or an injected one.
4. **The readout convention** — scalar eye position, or innervation of the `LR`/`MR` pair coupled to the soft-body eye.
5. **The target leak `xi_tau_s`** — a free parameter, or the quantity to be fitted against recorded eye position.

8. The eye: why eye F cannot yet do the task

Eye F is the MPM eye rebuilt from measured zebrafish anatomy — the six muscle attachments, the globe's flattening and each strap's width traced off the camera lucida in Tulenko & Currie (2020, fig 12.1A, after Easter & Nicola 1996), rather than from the mammalian textbook the earlier eyes A–E assumed. The anatomy is now

right and **the eye still cannot do the task**: driving each muscle alone to full command and holding it past settling gives a horizontal span of **7.9°** against the 15° the tracking task needs, and 6.3° vertical against 10°. Eye G, built from traced anatomy rather than the mammalian textbook, passes both at 15.9° and 24.2°.

Three families of parameter were swept on the lateral rectus to find out why — LR because it is the abductor, the muscle this circuit drives, and the one that sets the horizontal span. Every cell below is a *settled* measurement: the command is held 2.0 s, which is 1.5 settling times, and the pose is averaged over the last quarter with its peak-to-peak recorded beside it.

What was swept, and what it gave. *Geometry:* the sclera stand-off, 0.0161 → 0.080. This was the obvious suspect — the muscle and the globe are separate bodies that couple only through the shared MLS-MPM grid, so a strap lying hard against the sclera is welded along its whole arc of contact and drags the surface instead of winding the globe round. It is also the parameter that separates eye B from eye C. It is not the answer: travel *falls* monotonically, 6.11° → 1.08°. The B → C gain came from the strap *fraction* moving with it, 0.55 → 0.95, and F already runs 1.0. *Material:* active stress and stiffness together, 12 cells. Travel depends only on A/E, cleanly — two cells at A/E = 0.25 with quite different absolute values give 3.96° and 3.91° — and it saturates near A/E ≈ 0.3, above which the strap collapses on itself and the globe's radius drops 80%. F ships at 0.28, already at that edge. *Suspension:* fat, socket, and the resting pull of the five muscles that are not being driven. 6.10° → 7.06° with all three relaxed simultaneously, and the socket contributes exactly nothing — 6.104° in every cell of that sweep.

Why none of them works. LR shortens by **31% of its rest length** and its moment arm is **107 µm**, larger than the mammalian model's 69. If that shortening pulled the insertion round that arm the globe would turn 34.6°. It turns 6.1°. So **82% of the contraction is absorbed inside the muscle**: the measured fish straps are 10–20 µm wide against the mammalian model's 34, and a strap that slender buckles rather than transmits. That is a mechanical property of the model, not of the fish — the animal's muscle has an internal architecture the MPM strap does not.

Consequences for this document. First, the span gate in the characterisation protocol is not a formality: characterising F now would produce an exact description of an eye that cannot express the task. The levers left are anti-buckling — the `muscle_sleeve` that eyes D and E used and F has switched off — and cross-section. Second, **torsion is not negligible and the two-axis model of §5 needs revisiting**: at full command LR produces 3.86° of torsion against 6.17° of horizontal, and IO produces -11.08°. Both were measured on the same settled staircase runs, and the raw `curves.npz` for all six muscles are kept under `Plexus/prototype-/eye/archive/eye_F/`.

Appendix A. Which simulator, now that there are meshes

The question. A colleague is segmenting the six extraocular muscles and the globe from imaging, so for the first time we will have real anatomy rather than a hand-built approximation of it. Something has to consume those meshes and turn them into a moving eye. The realistic candidates are MuJoCo, the material-point simulator already in `Plexus/prototype/eye`, and a reimplementation of that simulator on NVIDIA's Warp.

The answer, first. Keep the material-point simulator. MuJoCo is faster, more mature and — importantly — already differentiable, which would remove the need for the fitted surrogate of section 5.3 entirely. But it cannot represent tissue that both deforms a lot and has different stiffness in different places, and that is precisely what a per-muscle segmentation is for. The medium-term move is to port the material-point simulator to Warp, which keeps the physics and adds the differentiability.

What the choice turns on. Not speed, and not accuracy in the abstract. It turns on what a segmented mesh is *for*. If the meshes are only there to fix where each muscle attaches and which way it pulls, a rigid globe with six cables wrapped around it uses all of that and runs a hundred times faster. If the meshes are there because the muscle belly, the tendon and the sclera behave differently and deform substantially while pulling, then the mesh is tissue, and only a simulator that carries material properties through the volume can use it. The second is the stated intent, so it decides the question.

	mesh becomes	different materials	large deformation	gradients	speed
MuJoCo, rigid globe + cable muscles	attachment points, wrap surfaces	not applicable	not applicable	yes (MJX / MuJoCo-Warp)	very fast
MuJoCo, deformable objects (`flex`)	tetrahedral volume	one per flex, or per edge	**no — small strain only**	yes	slow at high resolution
our material-point simulator	particles filling the volume	per particle	yes	**no**	slow
material-point on Warp	particles filling the volume	per particle	yes	yes, and batched	fast

MuJoCo with a rigid globe is the strongest option we are turning down, so it is worth being clear about what is being given up. It has a muscle model built in — the standard force-length-velocity relation, the same curve our thirty single-muscle holds exist to measure — so stage 1 of the protocol would simply not be needed. Its cables can be routed through wrap surfaces, which is a direct implementation of the connective-tissue pulleys Demer and colleagues established are real, and which eye D imitated with a sleeve constraint. And MuJoCo-Warp reports speedups of seventy to a hundred times with gradients available, so the eye could sit inside the training loop and be backpropagated through, rather than being replaced by a fitted stand-in. The cost is that the globe is rigid and the muscles are one-dimensional cables: the segmentation contributes geometry and nothing else, and every distinction between sclera, tendon and muscle belly is discarded on import.

MuJoCo's deformable bodies are the obvious way to keep the tissue, and they do not work here. First a word on the terms, because they mislead: a MuJoCo *body* is a rigid element of a kinematic tree and has no stiffness at all. Deformable objects are a separate element, `flex`, added in MuJoCo 3.0. MuJoCo is a rigid-body contact engine with deformation built on top, which is not a criticism — it is what it was designed for — but it is why continuum tissue sits awkwardly in it.

Two limits then apply. A flex carries **one** Young's modulus and one Poisson ratio for the whole object, so sclera, tendon and bone cap — 300, 9 and 40 in eye A — need three separate flexes fastened together, and joining elastic objects is reported to work only by giving each its own body. There is a partial escape: a flex is a mesh of edges, and edges carry stiffness and damping individually, so stiffness *can* be varied inside one flex at the edge level. But that means abandoning the continuum parameters and calibrating springs instead, which for a segmentation whose whole point is "this region is tendon and that region is sclera" is a downgrade rather than a solution.

More decisively, the deformation model is only valid for *small* strains, and warns that tetrahedra can invert under large load. A rectus muscle shortening by a third is not a small strain. That is a limitation of the method rather than of the implementation, so a newer version will not fix it — and it is consistent with the accuracy users report in large-deflection tests, where a beam expected to deflect 2.15 m settled at about 0.26 m. The rest of the implementation is improving quickly, for what it is worth: the solid elasticity model has moved from a plugin into the engine, and flex stiffness is now solved implicitly alongside contact rather than added as an external force.

The material-point method has the opposite profile, and it is the reason the prototype uses it. Material properties are carried by the particles, so every particle can differ and a mesh can be filled directly without being converted to tetrahedra first; and large deformation is the case the method was invented for, with no mesh to tangle. What it costs is everything section 4.7 already documents: the grid scatter overwrites its own memory, so the simulator cannot be differentiated, and it is far too slow to sit inside seven thousand gradient updates. That is the entire reason the surrogate exists.

The material-point method on Warp removes that cost without giving up anything above. Warp is a way of writing GPU code in Python with automatic differentiation built in, and material-point solvers written on it are public: `warp-mpm`, used in the PhysGaussian work, interoperates directly with PyTorch; GeoWarp is a differentiable implicit solver built specifically to fit material parameters by gradient descent; Rewarped runs many deformable simulations in parallel and returns gradients for all of them, which is the form training

actually needs, since a batch is a hundred and twenty-eight trials at once. Per-particle stiffness is the natural representation there — an array indexed by particle, with the gradient flowing back into it. Two honest caveats: this is a port rather than a setting, and GeoWarp demonstrates fitting one scalar parameter and describes spatially varying parameters as a generalisation rather than a result.

The plan, and the measurement that would change it. Near term, nothing changes: the material-point eye and the fitted surrogate of section 5.3, which work today and depend on no port. Medium term, port to Warp and put the eye itself in the training loop. The surrogate is not wasted either way — a differentiable simulator still has to be shown to reproduce the measured static map and mechanics, and the protocol's holds are exactly that test.

The one measurement that would overturn this is cheap. Build the MuJoCo version — rigid globe, six cables through wrap surfaces, the built-in muscle model — and run stage 0 and stage 1 on it. If it reproduces the measured span and the six single-muscle curves to within the accuracy the tracking task is scored at, then the deforming tissue is not buying anything the task can see, and we should take the hundredfold speedup and the free gradients. That is a day's work and it is worth doing before the port, not after.